ASSIGNMENT 1

Title: Evaluate and Justify the Operating-System Design of a Smart Hospital

STUDENT NAME	SIRANJEEVI G
REGISTER NUMBER	192572166
COURSE CODE	CSA0412
COURSE NAME	OPERATING SYSTEMS
TOTAL MARKS	50
NATURE OF ASSIGNMENT	INDIVIDUAL
GITHUB LINK	https://github.com/SIRANJEEVI192572166/OPERATING-SYSTEM_ASSIGNMENT.git

Scenario: Operating Systems for a Smart Hospital

Problem Statement

A 500-bed hospital is replacing its computer systems. It needs bedside patient monitors that raise an alarm within 10 milliseconds of an abnormal reading, nurses' handheld tablets, doctors' desktop workstations, a central patient-record database server that must never stop, an imaging server that processes MRI scans on many processors, and a link to two partner hospitals that share records. The IT team proposes one general-purpose operating system everywhere; the clinical staff fear delayed alarms, database stoppage during updates and tablet batteries that will not last a shift. This report evaluates each part of the system, judges the operating-system type and structure for each, examines system calls, and defends a complete plan. A Design Challenge (Task 12) then adds remote home monitoring and a cloud record service.

Table 1: Growth of the hospital system (from the scenario)

Stage	Bedside monitors	Tablets	Workstations
1 ward	40	–	–
1 hospital	500	120	80
Group of 3 hospitals (assumed 3 × hospital)	1,500	360	240

Execution Environment and Method

All programs in this report were compiled and executed on Linux. The environment is summarised in Table 2. Every measurement was produced by the programs listed in the report; the screenshots are the actual terminal output of those runs.

Table 2: Test environment

Item	**Value**
Linux distribution	Ubuntu 24.04.5 LTS
Kernel	Linux 6.18 (virtual machine, 1 CPU)
Compiler	gcc 13.3.0
Compile command	gcc -std=c11 -Wall -Wextra -pthread
System-call counting	sc.c, a small ptrace-based counter that prints in the strace -c format (strace itself was not installed on the test machine). The equivalent command on a machine with strace is: strace -c ./program

Note on measured values. Timings come from single runs on a one-CPU virtual machine and are indicative; the system-call counts are exact and repeatable. Figures marked “typical” or “assumed” in the analysis are engineering estimates stated with their assumptions.

TASK 1 – Evaluate the Requirements of Each Computing Environment

Bloom's Level: K5 – Evaluate. Each part of the hospital is evaluated for what it needs from its operating system.

1.1 Requirements of Each Device

Table 3: Response-time, reliability, throughput, battery and security requirements

Environment	**Response time**	**Reliability**	**Throughput**	**Battery**	**Security**
Bedside monitor	Alarm within 10 ms, guaranteed	Must run 24 h × 365 days	Low (a few samples/s)	Battery backup only for power cuts	Integrity of readings, authenticated network
Nurse tablet	Touch response < 100 ms	Moderate; restart allowed	Low	Critical: full 8–12 h shift	Encryption, lock, remote wipe
Doctor workstation	Interactive < 100 ms	Moderate	Medium (many applications)	Not needed (mains)	User login, role-based access
Database server	Query in milliseconds	Must never stop	High (3 hospitals)	UPS only	Confidentiality and integrity
Imaging server	Soft: minutes per scan	High: no lost scans	Very high (500 MB images)	UPS only	Protected image store
Partner link	Seconds acceptable	Retry, no lost updates	Medium (bandwidth)	Not applicable	Encryption, mutual authentication

1.2 Ranked Requirements and Justification

Table 4: Ranking for bedside patient monitors

Rank	Requirement	Justification
1	Guaranteed response time	A missed 10 ms alarm deadline is a patient-safety failure, so timing is more important than speed or features.
2	Reliability	The monitor must run continuously; a crash means an unmonitored patient.
3	Security	A forged or altered reading could cause a wrong clinical decision.
4	Battery	Mains-powered; the battery only bridges power cuts.
5	Throughput	Only a few samples per second are processed per patient.

Table 5: Ranking for nurses' handheld tablets

Rank	Requirement	Justification
1	Battery life	The tablet must last a whole shift; a flat tablet cannot be used at the bedside.
2	Security	Tablets carry patient data and can be lost or stolen.
3	Response time	Nurses need quick touch response but a delay of 100 ms is not dangerous.
4	Reliability	A restart is acceptable if data is synchronised with the server.
5	Throughput	Light workload: forms and record look-ups.

Table 6: Ranking for doctors' desktop workstations

Rank	Requirement	Justification
1	Security	Many users share the machine and view records; access must be controlled per user.
2	Response time	Doctors work interactively; the system must feel immediate.
3	Reliability	A crash costs time but patients are not endangered; the data is on the server.
4	Throughput	Several applications run together, which the CPU easily handles.
5	Battery	Mains powered, so not a design factor.

Table 7: Ranking for the central database server

Rank	Requirement	Justification
1	Reliability and availability	Records must be available at all times, including during updates.
2	Security	The database holds all confidential data and must remain correct.
3	Throughput	Hundreds of simultaneous transactions, growing to three hospitals.
4	Response time	Queries should return in milliseconds, but a short delay is not critical.
5	Battery	A UPS is used; no battery design is needed.

Table 8: Ranking for the MRI imaging server

Rank	Requirement	Justification
1	Throughput	Large images (hundreds of MB) must be processed quickly on many processors.
2	Reliability	A scan cannot be repeated cheaply; no result may be lost.
3	Security	Images are personal medical data.
4	Response time	Processing is batch-like; minutes are acceptable.
5	Battery	Not applicable.

Table 9: Ranking for the link to partner hospitals

Rank	Requirement	Justification
1	Security	Records cross organisational boundaries over a public network.
2	Reliability	Updates must not be lost or duplicated when the network fails.
3	Throughput	Bulk record exchange needs adequate bandwidth.
4	Response time	Delays of seconds are acceptable.
5	Battery	Not applicable.

1.3 Requirements That Conflict

Table 10: Conflicting requirements and their resolution

Conflict	**Where it occurs**	**Resolution**
Fast response versus many users on one machine	Monitor logic placed on a shared time-sharing server	Keep real-time work on a dedicated real-time system; never share it with users.
Throughput versus response time	Imaging server (big batch jobs) versus alarms	Run imaging on its own server; alarms never wait for image jobs.
Security versus battery and speed	Encryption and full-disk protection on tablets	Use hardware-assisted encryption; accept a small battery cost.
Never stop versus frequent updates	Database server patching	Cluster with fail-over: update one node at a time.
Rich services versus small and fast	Full desktop OS on tablets or monitors	Use a reduced OS with only the necessary services.

1.4 Requirements Critical to Patient Safety

- **Bounded alarm latency** (10 ms) on the bedside monitors: the only requirement with a hard deadline.
- **Continuous availability** of the monitors and the database: records and alarms are needed during emergencies.
- **Data integrity and access control**: wrong or altered records can cause wrong treatment, and leaks breach privacy.
- **Tablet battery life**: a dead tablet at the bedside removes access to medication records.

1.5 Lessons from the History of Operating Systems

Table 11: How OS history applies to each environment

Historical stage	Lesson	Hospital environment
Batch systems	Run long jobs to completion, queue them, maximise throughput	Imaging server (MRI processing jobs)
Time-sharing	Share the CPU in small time slices so many users get fast interactive response	Database server and shared workstations
Personal computers	Single-user machine, graphical interface, low cost	Doctors' desktop workstations
Mobile systems	Power management, touch interface, sandboxed applications	Nurses' tablets
Real-time embedded systems	Predictable timing matters more than average speed	Bedside monitors

1.6 Alternative Concept and Complexity Analysis: One General-Purpose OS Everywhere

If a single general-purpose time-sharing OS ran on every device, the worst-case alarm delay would depend on the scheduler. With N runnable tasks and a time quantum q, a task may wait about $(N - 1) \times q$ before it runs. For $N = 20$ and $q = 4$ ms this is 76 ms, far above 10 ms, and page faults, kernel locks and background services make the bound unpredictable. A real-time OS with fixed-priority pre-emption guarantees: worst-case delay = interrupt latency + context-switch time + handler time, typically well below 1 ms and independent of N.

Table 12: Single general-purpose OS compared with one OS type per environment

Measure	**One general-purpose OS everywhere**	**Different OS type per environment**
Worst-case alarm response	Unbounded (about (N−1) × q, e.g. 76 ms, plus page-fault and lock delays) – time complexity O(N)	Guaranteed deadline, below 1 ms, independent of load – O(1)
Memory per bedside monitor (typical)	1–4 GB RAM and several GB storage	64 KB – 2 MB RAM; an RTOS kernel such as FreeRTOS needs only a few KB
Memory for 500 monitors (typical)	500 × about 2 GB = about 1,000 GB	500 × about 1 MB = about 500 MB
Administration effort	One image, one patch process, one skill set (lowest)	4–5 OS families, more patch processes and skills (higher)
Patient-safety verdict	Fails the 10 ms requirement; rejected for monitors	Meets every requirement; accepted

Evidence provided: ranked requirement tables for every environment (the six ranking tables above), the conflict table and the single-OS comparison above.

TASK 2 – Judge the Type of Operating System for Each Environment

Bloom's Level: K5 – Evaluate. The types of operating system are compared and matched to the hospital.

2.1 Types of Operating Systems Compared

Table 13: Types of operating systems and their suitability

Type	**Main property**	**Suitable hospital use**
Mainframe	Very high I/O throughput and reliability; batch and transaction processing	Possible for a central records system, but costly
Desktop	One interactive user, GUI, general applications	Doctors' workstations
Multiprocessor (parallel)	Several CPUs share one memory and one OS	Imaging server
Distributed	Separate computers appear as one system over a network	Partner-hospital record sharing
Clustered	Several computers share storage and take over for each other	Database server with fail-over
Hard real-time	Deadline is always met; failure of a deadline is a system failure	Bedside monitors (10 ms alarm)
Soft real-time	Deadlines are met on average; occasional misses tolerated	Video/streaming of scans on workstations
Handheld	Small, battery powered, touch interface, limited memory	Nurses' tablets

2.2 Chosen Type for Each Environment

Table 14: Justified choice of OS type with rejected alternatives

Environment	**Chosen type**	**Rejected alternatives and reasons**
Bedside monitors	Hard real-time OS	General-purpose OS: no guaranteed 10 ms. Soft real-time: a missed alarm is unacceptable.
Nurses' tablets	Handheld (mobile) OS	Desktop OS: too much memory, power and start-up time.
Doctors' workstations	Desktop OS	Handheld OS: limited multitasking and screen. Mainframe terminal: unnecessary cost.
Database server	Clustered system with fail-over	Single machine: one failure stops records. Mainframe: very costly.
Imaging server	Symmetric multiprocessing OS	Asymmetric: master CPU becomes a bottleneck. Cluster: image data would be moved between nodes.
Partner-hospital link	Distributed system / network gateway	Single local system: cannot share records across sites safely.

2.3 Symmetric versus Asymmetric Multiprocessing for the Imaging Server

Table 15: Symmetric and asymmetric multiprocessing

Aspect	**Symmetric (SMP)**	**Asymmetric (AMP)**
Organisation	Every CPU runs the same OS and may run any task	One master CPU runs the OS; slave CPUs run assigned tasks
Load balancing	Automatic, any CPU takes the next job	Master assigns work and can become a bottleneck
Failure of a CPU	System continues with fewer CPUs	Master failure stops the system
Scalability	Good for 8–64 CPUs (shared memory limits growth)	Poor as slaves increase
Fit for MRI processing	Best: many image slices processed in parallel	Not chosen

2.4 Clustered Database Server with Fail-over

Two database nodes share mirrored storage. Node A is active and serves all queries. Node B is standby and watches Node A through a heartbeat message. If the heartbeat stops, Node B takes over the shared disks and the service address within seconds, so clients reconnect without losing committed records.

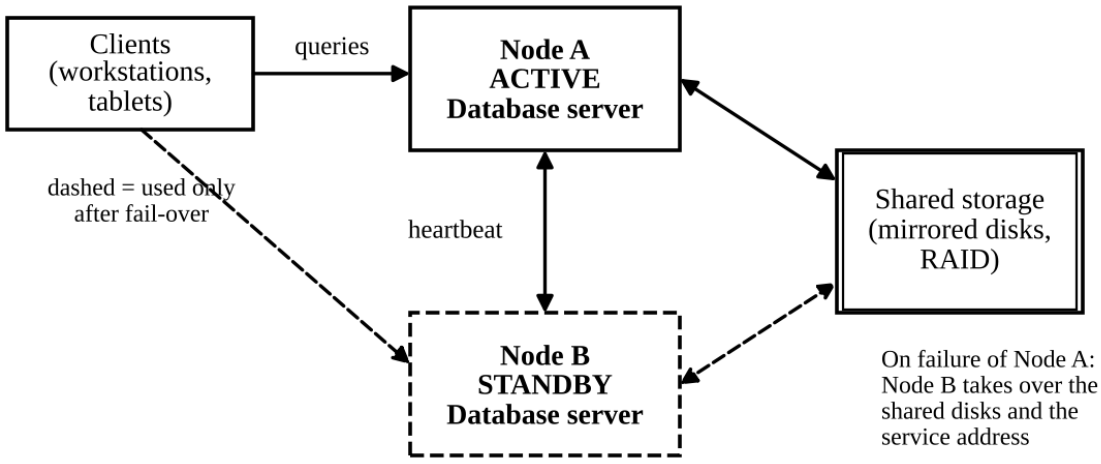


Figure 1: Clustered database server with fail-over

2.5 Alternative Concept and Complexity Analysis: Single Large Multiprocessor Database Server

Availability (time). A single large machine is a single point of failure. If its availability is 99.9 percent, downtime is about 8.76 hours per year. In a two-node cluster, service stops only if both nodes fail together: $1 - (1 - 0.999)^2 = 99.9999$ percent, or about 31.5 seconds per year (ignoring the few seconds of fail-over time).

Throughput growth. By Amdahl's law, speed-up $S(p) = 1 / ((1 - f) + f / p)$, where f is the parallel fraction. For $f = 0.95$:

Table 16: Speed-up of the single multiprocessor machine (f = 0.95)

Processors p	**2**	**4**	**8**	**16**	**32**	**64**	∞
Speed-up S(p)	1.90	3.48	5.93	9.14	12.55	15.42	20

Throughput grows with p only until the serial fraction dominates (maximum $1/(1 - f) = 20$), so doubling the CPUs from 32 to 64 gives only 23 percent more speed. Complexity of the speed-up is O(1) in the limit, not O(p).

Table 17: Single large multiprocessor compared with a cluster with fail-over

Criterion	**Single large multiprocessor**	**Cluster with fail-over**
Availability	Single point of failure (about 99.9 percent, 8.76 h/year)	About 99.9999 percent, fail-over in seconds
Cost	Cost rises faster than capacity (specialised hardware)	Two commodity nodes plus shared storage
Throughput growth	Limited by Amdahl's law and shared-memory contention	Standby is idle; scale-out needs extra nodes and data partitioning
Memory (space)	One large shared memory visible to all CPUs	Each node has its own RAM; replicated data uses memory twice
Updates	Whole machine stops during major updates	Rolling update: one node at a time

Decision: the cluster with fail-over is retained for the database because the requirement "must never stop" ranks above raw throughput (Task 1).

Evidence provided: a justified choice for each environment with rejected alternatives (the choice table above), SMP/AMP comparison and the cluster and availability analysis.

TASK 3 – Evaluate Operating-System Structures

Bloom's Level: K5 – Evaluate. The possible structures of the operating system are evaluated against the hospital requirements.

3.1 The Five Structures

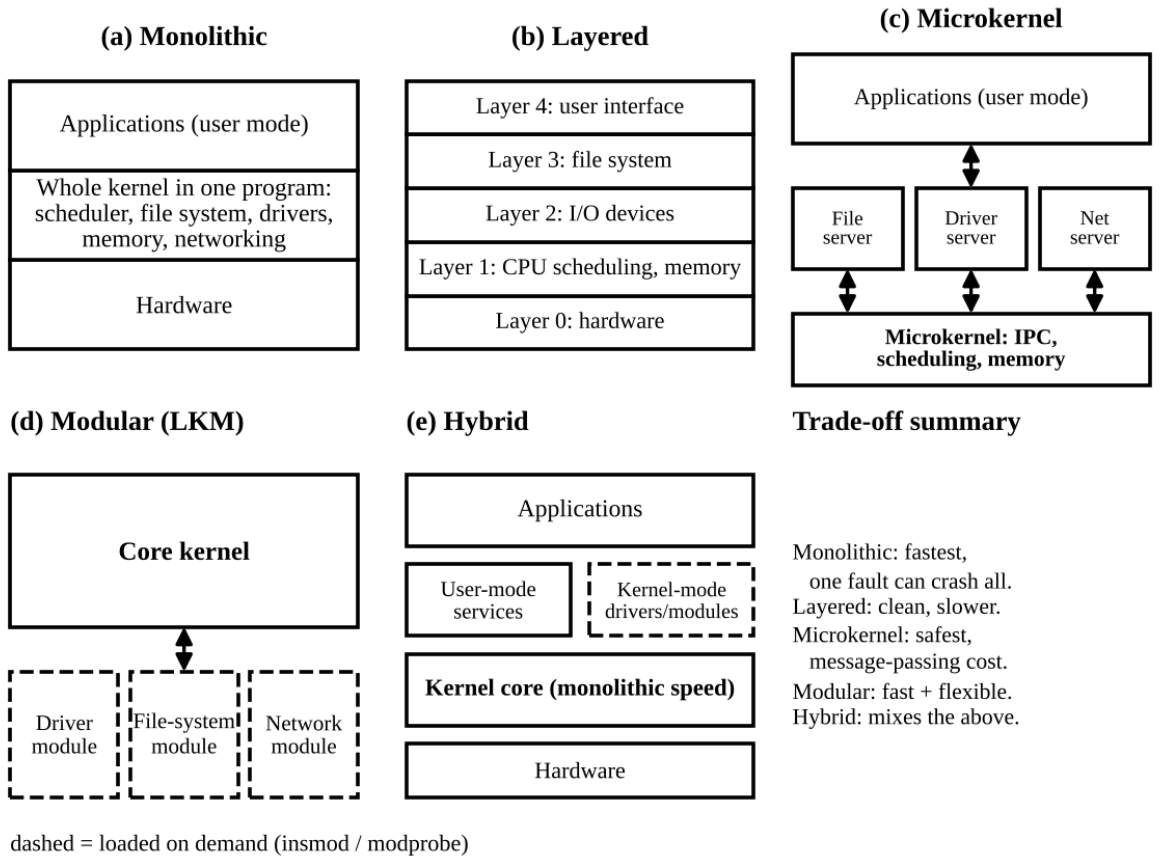


Figure 2: Operating-system structures

Table 18: Effect of each structure on reliability, performance and ease of updating

Structure	**Reliability**	**Performance**	**Ease of updating**
Simple / monolithic	Low: a fault in any driver can crash the kernel	Highest: calls are direct function calls	Hard: whole kernel is rebuilt
Layered	Good: errors are confined to layers	Lower: every call crosses several layers	Medium: a layer can be replaced if its interface is kept
Microkernel	Highest: drivers and servers run in user mode and can be restarted	Lower: services use message passing	Easy: a server is replaced without rebooting
Modular (loadable modules)	Medium: modules run in kernel mode, so a faulty module can still crash it	High: calls between modules are direct	Easy: modules are loaded and removed on demand
Hybrid	Medium to high	High	Medium

3.2 Suitable Structure for the Monitors and for the Database Server

Table 19: Defended choice of structure for two environments

Environment	**Chosen structure**	**Reason**
Bedside monitors	Microkernel RTOS (for example QNX) or a minimal RTOS kernel (FreeRTOS)	Small kernel, predictable timing, and a faulty driver or service can be restarted without stopping the alarm task.
Database server	Modular monolithic kernel (Linux with loadable modules, live patching)	Highest throughput for file and network I/O; drivers and file systems can be updated without full reinstallation; the cluster covers kernel failures.

3.3 Structure of Real Operating Systems

Table 20: Structure of Linux, Windows, Android and real-time systems

Operating system	**Structure**
Linux	Monolithic kernel with loadable kernel modules: scheduler, memory manager, file systems and drivers run in one address space in kernel mode.
Windows	Hybrid: a kernel (scheduling, interrupts) under an executive layer (I/O manager, memory, security), with drivers in kernel mode and subsystem services in user mode.
Android	Layered: Linux kernel, hardware abstraction layer, native libraries and runtime, application framework, applications.
FreeRTOS	Minimal library-style kernel (scheduler, queues, semaphores, timers) linked with the application in one address space.
QNX	Microkernel: only scheduling, IPC and basic memory in the kernel; file system, drivers and networking are user-mode processes.

3.4 Alternative Concept and Complexity Analysis: Monolithic Kernel on the Bedside Monitors

Time cost of a service request. In a monolithic kernel a service request is one system call: one trap, run the kernel function, return, so the cost is constant, O(1). In a microkernel a request passes through k servers (for example file server then driver server), costing 2k messages, so the cost is O(k). The cost of both paths was measured with the program bench_cost.c: a system call (getpid) and a round trip of two messages between two processes through pipes (see Task 6).

Table 21: Measured cost of a system call and of message passing

Operation	**Measured time**	**Ratio**
One system call (monolithic request)	100.6 ns	1
One IPC round trip (microkernel request through one server)	2,470.9 ns	24.6 × one system call

Memory taken by the kernel (space). A typical monolithic general-purpose kernel image is 10–20 MB in memory with its drivers; a microkernel such as QNX has a kernel of tens of KB, plus only the servers that are needed.

Effect of a faulty driver. In a monolithic kernel a faulty driver runs in kernel mode and can overwrite kernel data, which halts the whole monitor and its alarm. In a microkernel the driver is an ordinary process, so a fault only kills that process, which is restarted in milliseconds while the alarm task continues.

Table 22: Monolithic kernel compared with microkernel for bedside monitors

Criterion	**Monolithic kernel**	**Microkernel**
Request cost	O(1), about 0.1 µs (measured)	O(k) messages, about 2.5 µs per server (measured)
Kernel memory	Large (about 10–20 MB)	Small (tens of KB)
Faulty driver	Whole system fails	Only the driver process fails and restarts
Verdict	Faster but unsafe for alarms	Chosen: the extra microseconds are far below the 10 ms limit

The microkernel's extra cost is about 2.5 µs per request, which is 0.025 percent of the 10 ms deadline, so reliability is gained at negligible timing cost.

Evidence provided: comparison of structures against the hospital requirements with a defended choice for two environments (the structure choice table above).

TASK 4 – Evaluate the Services an Operating System Must Provide

Bloom's Level: K5 – Evaluate. The services each environment needs are evaluated.

4.1 Operating-System Services

Table 23: Services of an operating system

Service	**Purpose**
User interface	Command line, GUI or touch interface to use the system
Program execution	Load a program into memory, run it and end it
I/O operations	Access to disks, sensors, network cards without the program handling the hardware
File-system manipulation	Create, read, write, delete and protect files and directories
Communication	Message passing and shared memory between processes and computers
Error detection	Detect and handle hardware, I/O and program errors
Resource allocation	Share CPU, memory and devices among programs
Accounting	Record which user uses how much of each resource
Protection and security	Control access to resources and authenticate users

4.2 Service Table for Each Environment

Table 24: Services included, reduced or excluded in each environment

Service	Monitor	Tablet	Workstation	Database server	Imaging server
User interface	Excluded (fixed display)	Included (touch)	Included (GUI)	Reduced (remote shell)	Reduced (remote shell)
Program execution	Included (fixed tasks)	Included	Included	Included	Included
I/O operations	Included (sensor, alarm)	Included	Included	Included	Included (DMA)
File systems	Reduced (flash only)	Included	Included	Essential	Essential
Communication	Included (network)	Included	Included	Included	Included
Error detection	Essential	Included	Included	Essential	Included
Resource allocation	Reduced (static)	Included	Included	Essential	Essential
Accounting	Excluded	Excluded	Reduced	Included (audit)	Included
Protection and security	Included (integrity)	Included (encryption)	Included (login)	Essential	Included

Services removable from the bedside monitor to make it smaller and faster: the graphical user interface, multi-user login and accounting, a general file system, printing, background update and indexing daemons, and dynamic loading of arbitrary programs. Every removed service reduces code size, start-up time, attack surface and timing jitter.

Services essential for the database server: file-system and storage management with journalling, resource allocation (CPU, memory and I/O scheduling), protection and security, error detection and recovery, communication (network), and accounting for audit.

4.3 How the OS Protects One Patient's Records from Another User's Programs

- **Authentication:** each user logs in with a user identifier and credentials.
- **Memory protection:** the hardware prevents one process from reading another process's address space (Task 8).
- **File permissions and access-control lists:** the file system checks the user and group on every open() call.
- **Privileged operations only in kernel mode:** programs reach records only through system calls that check rights.
- **Encryption and audit logs:** records are encrypted at rest and every access is logged.

4.4 Alternative Concept and Complexity Analysis: Full Desktop OS on the Tablets

With every service enabled, the tablet carries many extra background services. Typical figures used for the analysis (assumed; a 30 Wh battery): a reduced handheld OS uses about 3 W in use, a full desktop OS about 6.5 W.

Table 25: Reduced handheld OS compared with full desktop OS on a tablet (typical, assumed values)

Measure	**Reduced handheld OS**	**Full desktop OS**
Memory in use at start (space)	About 1.5 GB	About 4 GB
Installed storage (space)	About 6 GB	About 25 GB
Start-up time (time)	About 10 s	About 45 s
Battery life = 30 Wh / power	30 / 3 = 10 hours	30 / 6.5 = 4.6 hours
Covers a 10-hour shift?	Yes	No

Battery life is inversely proportional to the power drawn, so extra services that keep the CPU awake reduce life from 10 to about 4.6 hours. The full desktop OS is therefore rejected for the tablets.

Evidence provided: a service table for every environment with justified inclusions and exclusions (the service table above).

TASK 5 – Examine System Calls

Bloom's Level: K5 – Evaluate. This task examines how a program obtains operating-system services.

5.1 Categories of System Calls

Table 26: Categories of system calls with Linux examples

Category	**Purpose**	**Examples**
Process control	Create, run, end and wait for processes	fork, execve, exit, waitpid, brk
File management	Create, open, read, write, close and delete files	open, read, write, close, unlink, lseek
Device management	Request, use and release devices	ioctl, read and write on device files, mmap
Information maintenance	Get or set system and process data	getpid, gettimeofday, uname, sysinfo
Communication	Message passing and shared memory	pipe, socket, send, recv, shmget
Protection	Control access to resources	chmod, chown, umask, setuid

5.2 What Happens When a Program Calls read()

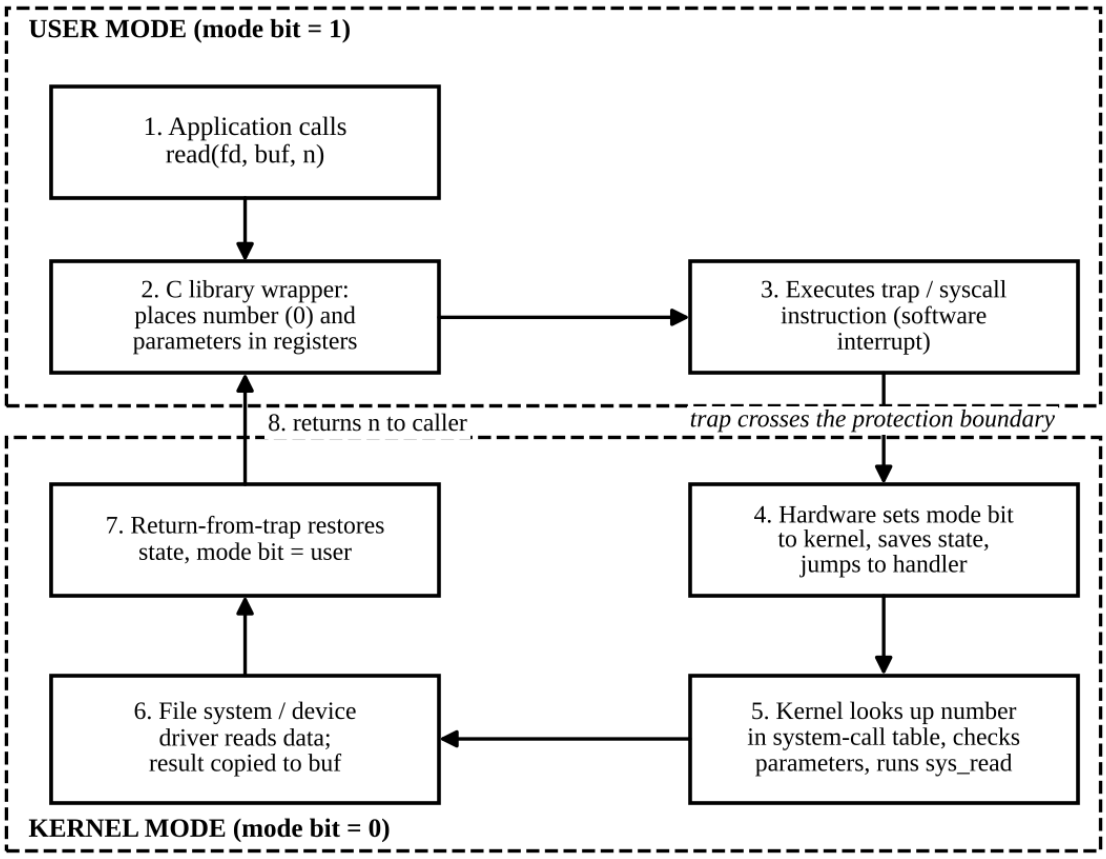


Figure 3: Steps of the read() system call from user mode to kernel mode and back

Table 27: Step-by-step execution of read(fd, buf, n)

Step	Action	Mode
1	The application calls read(fd, buf, n), an ordinary function call into the C library	User
2	The library wrapper places the system-call number (0 for read on x86-64) in a register and the parameters in other registers	User
3	The wrapper executes the syscall (trap) instruction	User to kernel
4	The hardware switches the mode bit to kernel mode, saves the user program counter and jumps to the kernel entry point	Kernel
5	The kernel uses the number to index the system-call table and checks the parameters (valid fd, valid buffer address)	Kernel
6	The file system or driver gets the data (from the page cache or the device) and copies it to the user buffer	Kernel
7	The kernel places the return value (bytes read or −1) in a register and executes the return-from-trap instruction	Kernel to user
8	The mode bit is set back to user mode and the program continues after the call	User

5.3 Passing Parameters to the Kernel

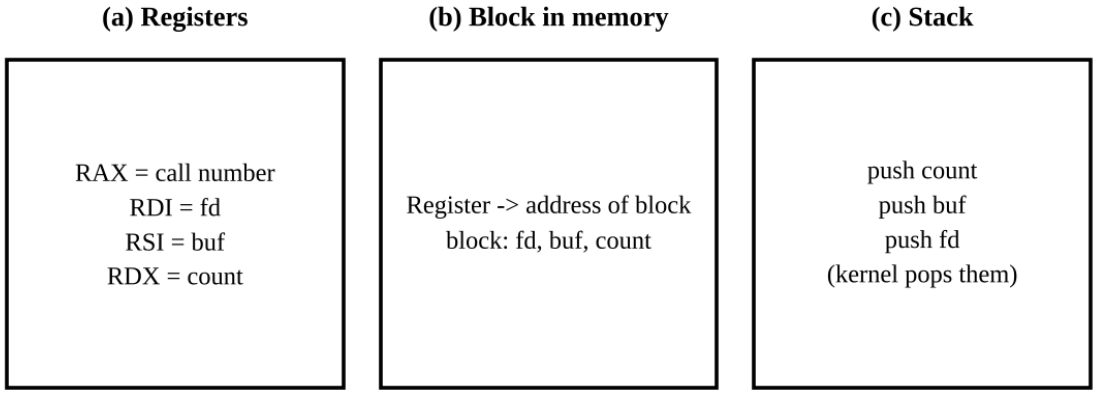


Figure 4: Three methods of passing parameters to the kernel

Registers are fastest and used on Linux x86-64 for up to six parameters. When there are more parameters than registers, or the data is large, a block in memory is used and its address is

passed in a register. The stack method is used on some systems and is pushed by the program and popped by the operating system.

5.4 Using strace to See System Calls

strace runs a program and prints every system call with its parameters and return value; strace -c prints a summary count of calls. The program q5_trace copies the 10,000-byte file with a 4096-byte buffer. Because strace was not installed on the test machine, the trace was produced by sc -t, a small ptrace-based tracer (written for this report) that prints the same name(argument) = result lines; it shows only the file-related calls and prints "..." in place of buffer contents. The equivalent command on a machine with strace is strace -e trace=openat,read,write,close ./q5_trace.

Terminal - Task 5: system-call trace of q5_trace (buffer = 4096)

```
$ ./sc -t ./q5_trace
openat(AT_FDCWD, "records.dat", 0x241) = 3
write(3, "...", 4096) = 4096
write(3, "...", 4096) = 4096
write(3, "...", 1808) = 1808
close(3) = 0
openat(AT_FDCWD, "records.dat", 0x0) = 3
openat(AT_FDCWD, "copy.dat", 0x241) = 4
read(3, "...", 4096) = 4096
write(4, "...", 4096) = 4096
read(3, "...", 4096) = 4096
write(4, "...", 4096) = 4096
read(3, "...", 4096) = 1808
write(4, "...", 1808) = 1808
read(3, "...", 4096) = 0
close(3) = 0
close(4) = 0
unlink("records.dat") = 0
unlink("copy.dat") = 0
buffer 4096 bytes: 4 read() calls
write(1, "...", 34) = 34
```

Figure 5: System-call trace of the copy program with a 4096-byte buffer

The trace begins with the C library creating the test file (openat for records.dat, three writes of 4096, 4096 and 1808 bytes, close). The copy then opens the source (descriptor 3) and the destination (descriptor 4) with openat and loops: read(3, ..., 4096) followed by write(4, ..., 4096) twice, a short read of 1808 bytes followed by a write of 1808 bytes, and a final read that returns 0 (end of file). The program then closes both descriptors and removes the two files with unlink. So 4 read() calls are needed for 10,000 bytes, which agrees with the expected output.

5.5 Alternative Concept and Complexity Analysis: fopen, fread and fwrite Instead of open, read and write

The C library functions keep an internal buffer (4096 bytes for a regular file with glibc). A call to fread for one byte is satisfied from this buffer, and a system call is made only when the buffer is empty or full. For a file of n bytes:

- Direct read() with a buffer of b bytes: about n / b system calls (n calls when b = 1). Time cost O(n / b).
- Library with 4096-byte internal buffer: about n / 4096 system calls, whatever size the program's own fread requests are. Time cost O(n / 4096), at the price of one extra buffer of 4096 bytes per open file (space O(1)) and one extra memory copy.

Terminal - Task 5 alternative: fread/fwrite with 1-byte calls (10,000-byte file)

```
$ ./sc ./copy_stdio small.dat
fread/fwrite copy: 10000 bytes processed with 1-byte library calls
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 12.91    0.000023           6         4           write
  7.76    0.000014           3         5           read
  2.33    0.000004           2         2           pread64
------ ----------- ----------- --------- --------- ----------------
100.00    0.000176                    49         1 total
```

Figure 6: Library copy (fread/fwrite one byte at a time) of the 10,000-byte file

Although the program made 10,000 fread() calls and 10,000 fwrite() calls, the summary shows only 4 read() and 4 write() system calls, of which one read belongs to the program loader and one write to the printed message. The same file copied with one-byte read() calls needed 10,001 read() calls (Task 6).

Table 28: Direct system calls compared with the C library for a 10,000-byte file

Method	read() system calls	Extra buffer space
read() with 1-byte buffer	10,001	0
read() with 4096-byte buffer	4	4096 bytes (program)
fread() of 1 byte at a time	4 (library buffer 4096)	4096 bytes (library)

Evidence provided: a step-by-step explanation of the read() system call with the diagram, and the strace-style evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls

Bloom's Level: K5 – Evaluate. This task evaluates how the number of system calls affects performance.

6.1 Why a System Call Costs More than a Function Call

A function call only saves the return address and jumps. A system call must switch the CPU from user mode to kernel mode, save and restore the user registers, switch to the kernel stack, validate parameters, run the kernel function, and return. It also disturbs the instruction and data caches and the TLB and may need security mitigations. The cost was measured with bench_cost.c.

```
Terminal - Task 6: cost of a function call, a system call and IPC

$ ./bench_cost
ordinary function call        :      0.4 ns
system call (getpid)          :    100.6 ns
IPC round trip (2 pipes)      :   2470.9 ns
system call / function call   :    232.7 x
IPC round trip / system call  :     24.6 x
```

Figure 7: Measured cost of a function call, a system call and an IPC round trip

A system call (100.6 ns) cost about 233 times an ordinary function call (0.4 ns, the figure also includes loop overhead). Millions of unnecessary system calls therefore directly waste CPU time.

6.2 Complete the Code

TODO 1 of copy_file() was completed with a loop that reads up to bufsize bytes, counts every read call, stops when read returns 0 or −1, and writes exactly the n bytes that were read. Two safety additions were made inside the TODO region only: the buffer size is limited to the 8192-byte array, and a partial write is repeated until all n bytes are written. The rest of the program is unchanged.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
```

```
long copy_file(const char *src, const char *dst, size_t bufsize)
{
   char buf[8192];
   int in = open(src, O_RDONLY);
   if (in < 0) return -1;
   int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
   if (out < 0) { close(in); return -1; }
   long calls = 0;
   ssize_t n;
   // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
   //  stop when read returns 0 (end of file) or -1; write exactly the n bytes that
   //  were read
   if (bufsize > sizeof buf) bufsize = sizeof buf;  /* never overflow buf */
   while ((n = read(in, buf, bufsize)) > 0) {
      calls++;  /* count every successful read */
      ssize_t done = 0;
      while (done < n) {  /* write exactly n bytes */
         ssize_t w = write(out, buf + done, (size_t)(n - done));
         if (w < 0) { close(in); close(out); return -1; }
         done += w;
      }
   }
   calls++;  /* the final read that returned 0 or -1 */
   if (n < 0) calls = -1;  /* read error */

   close(in);
   close(out);
   return calls;
}

int main(void)
{
   FILE *f = fopen("records.dat", "w");  /* a 10,000-byte test file */
   for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
```

```
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

The program was compiled with gcc -std=c11 -Wall -Wextra -pthread and executed. The output is identical to the expected output:

Terminal - Task 5/6: build and run q5_copy

```
$ gcc -std=c11 -Wall -Wextra -pthread -o q5_copy q5_copy.c
$ ./q5_copy
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Figure 8: Compilation and execution of the completed copy program

6.3 Comparison with strace -c

Each run of the program was counted with the strace -c style tool and compared with the number reported by the program.

Terminal - Task 5: strace -c style summary, buffer = 1

```
$ ./sc ./q5_one 1
program reports: buffer    1 bytes: 10001 read() calls
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 50.76    0.024013           2     10004           write
 48.80    0.023088           2     10002           read
  0.01    0.000004           2         2           pread64
------ ----------- ----------- --------- --------- ----------------
100.00    0.047311                 20047         1 total
```

Figure 9: System-call summary for a 1-byte buffer

Terminal - Task 5: strace -c style summary, buffer = 512

```
$ ./sc ./q5_one 512
program reports: buffer  512 bytes:    21 read() calls
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 26.86    0.000077           3        24           write
 17.03    0.000049           2        22           read
  1.52    0.000004           2         2           pread64
------ ----------- ----------- --------- --------- ----------------
100.00    0.000288                    87         1 total
```

Figure 10: System-call summary for a 512-byte buffer

Terminal - Task 5: strace -c style summary, buffer = 4096

```
$ ./sc ./q5_one 4096
program reports: buffer 4096 bytes:     4 read() calls
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 17.35    0.000034           5         7           write
  5.98    0.000012           2         5           read
  2.18    0.000004           2         2           pread64
------ ----------- ----------- --------- --------- ----------------
100.00    0.000198                    53         1 total
```

Figure 11: System-call summary for a 4096-byte buffer

Table 29: Number of read() calls: program output compared with the system-call counter

Buffer size	Program output	Counter shows	Difference
1 byte	10001	10002	+1
512 bytes	21	22	+1
4096 bytes	4	5	+1

The counter always shows one more read() than the program because the dynamic loader of the C library performs one read() of the ELF header when the program starts; the program's own count (10,000 + 1 end-of-file read for the 1-byte case) is exactly matched. The write() counts exceed the data writes for similar reasons: the test file is created with three buffered writes by the C library and one message is printed.

6.4 Effect of Buffer Size on the Database Backup

The number of read() and write() calls for an n-byte file is ⌈n / b⌉ each, so doubling the buffer halves the calls. A 4 MB file was copied with bench_copy.c using seven buffer sizes (time measured inside the program with a monotonic clock).

```
Terminal - Task 10: copying a 4 MB file with different buffer sizes (time)

$ for b in 1 16 64 512 4096 65536 1048576; do time ./bench_copy big4m.dat $b; done
bufsize       1 :   4194304 read() calls,   1757.1 ms  real 0m1.759s user 0m0.402s sys 0m1.336s
bufsize      16 :    262144 read() calls,     89.7 ms  real 0m0.091s user 0m0.021s sys 0m0.070s
bufsize      64 :     65536 read() calls,     27.8 ms  real 0m0.030s user 0m0.008s sys 0m0.020s
bufsize     512 :      8192 read() calls,      6.5 ms  real 0m0.009s user 0m0.000s sys 0m0.007s
bufsize    4096 :      1024 read() calls,      2.0 ms  real 0m0.004s user 0m0.003s sys 0m0.000s
bufsize   65536 :        64 read() calls,      1.9 ms  real 0m0.004s user 0m0.000s sys 0m0.003s
bufsize 1048576 :         4 read() calls,      1.3 ms  real 0m0.003s user 0m0.000s sys 0m0.002s
```

Figure 12: Measured copy of a 4 MB file with different buffer sizes

Table 30: Buffer size, read() calls and time for a 4 MB file

Buffer (bytes)	read() calls	Time (ms)	Speed-up over 1 byte
1	4,194,304	1757.1	1.0
16	262,144	89.7	19.6
64	65,536	27.8	63.2
512	8,192	6.5	270
4,096	1,024	2.0	879
65,536	64	1.9	925
1,048,576	4	1.3	1,352

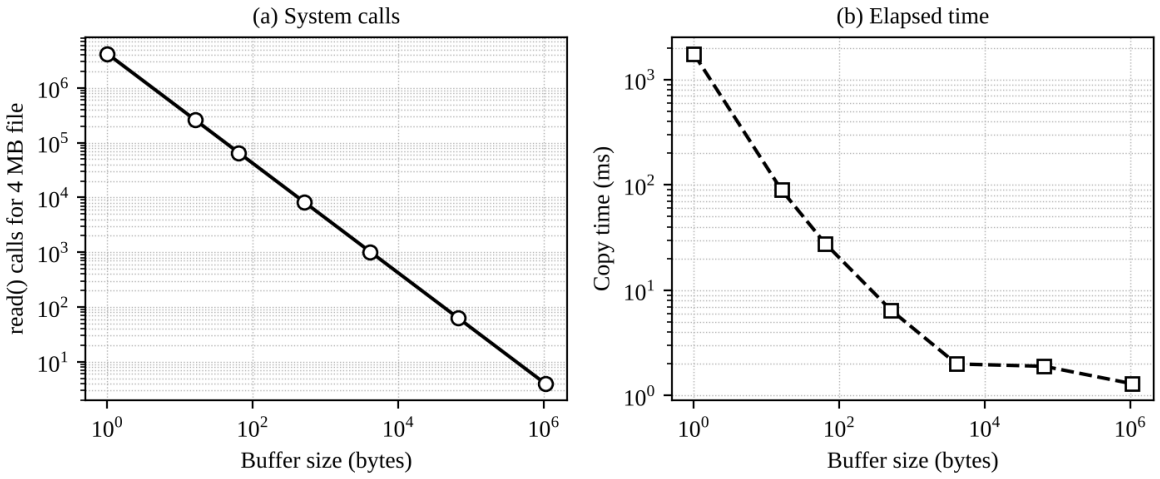


Figure 13: Number of system calls and copy time against buffer size (log scales)

The call count falls exactly as 1/b (measured counts equal 4,194,304 / b). The time falls steeply up to about 4 KB and then flattens, because beyond that point the system-call overhead is small compared with the cost of copying the data itself.

6.5 Library Buffering

fread and fwrite hide the system-call cost: the library groups many small requests into a few large read() and write() calls (Task 5). A program that reads one byte at a time with fread therefore costs about the same number of system calls as one that uses a 4096-byte buffer.

6.6 Recommended Buffer Size for the Hospital's Backup Program

Recommendation: 64 KiB (65,536 bytes). The measurements show that 4096 bytes already gives a 879-fold gain over 1 byte, and 64 KiB reduces calls from 1,024 to 64 for a 4 MB file with the lowest time before the curve flattens (1.9 ms). 64 KiB is small enough for any server (negligible memory, fits in the cache) and a multiple of the 4096-byte disk page, so I/O is aligned. Larger buffers such as 1 MiB reduce time only slightly (1.3 ms for this small file) and increase memory use per concurrent backup. These timings come from single runs on a small file in the page cache; on a real disk the benefit of large buffers is greater, so 64 KiB to 1 MiB is a safe range.

6.7 Alternative Concept and Complexity Analysis: Memory-Mapped Files (mmap)

With mmap the file is mapped into the program's address space and copied with memcpy. The program makes a few system calls (open, fstat, ftruncate, mmap × 2, munmap × 2, close), independent of the file size, instead of about n / b read and write calls.

Terminal - Task 6 alternative: mmap copy (10,000-byte file)

```
$ ./sc ./copy_mmap small.dat
mmap copy: 10000 bytes
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 26.96    0.000043           4        10           mmap
  2.66    0.000004           2         2           pread64
  2.27    0.000004           4         1           write
  1.45    0.000002           2         1           read
------ ----------- ----------- --------- --------- ----------------
100.00    0.000160                    46         1 total
```

Figure 14: System-call summary of the mmap copy of a 10,000-byte file

```
Terminal - Task 6 alternative: 4 MB file, stdio (1-byte calls) and mmap
$ time ./copy_stdio big4m.dat
$ time ./copy_mmap big4m.dat
fread/fwrite copy: 4194304 bytes processed with 1-byte library calls

real 0m0.079s
user 0m0.078s
sys 0m0.000s
mmap copy: 4194304 bytes

real 0m0.003s
user 0m0.003s
sys 0m0.000s
```

Figure 15: Time to copy a 4 MB file with one-byte library calls and with mmap

Table 31: read/write with a buffer compared with mmap

Criterion	**read() and write() with buffer**	**mmap and memcpy**
System calls (time)	About 2n / b, O(n / b)	A constant few, O(1)
Copying between kernel and user space	Data is copied twice: kernel to user buffer, user buffer to kernel	No copy to a user buffer; the pages of the page cache are mapped directly (one memcpy between mapped pages)
Page faults	None per call	About n / 4096 page faults, each handled by the kernel, so the overhead moves from calls to faults
Memory for very large files (space)	Only the buffer, O(b)	Virtual address space of size n (needs 64-bit); physical memory only for pages touched
Measured, 4 MB	2.0 ms (4096-byte buffer)	3 ms (including process start); one-byte library calls: 79 ms

mmap is the better design for very large backup files on a 64-bit server, while a 64 KiB buffer is simplest and nearly as fast for ordinary files.

Evidence provided: measured system-call counts and times for different buffer sizes with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls (4 Marks)

Bloom's Level: K5 – Evaluate. This task evaluates how a command shell on the database server starts and manages programs.

7.1 fork(), exec(), wait() and exit()

Table 32: Process control system calls

Call	What it does
fork()	Creates a new child process that is a copy of the parent (copy-on-write memory). Returns the child's PID to the parent and 0 to the child.
exec() family (execlp, execve)	Replaces the program in the calling process by a new program; the PID stays the same. Returns only if it fails.
wait() / waitpid()	Blocks the parent until a child ends and returns the child's exit status; the kernel then frees the child's process-table entry.
exit() / _exit()	Ends the process and passes an exit status (0 = success) to the parent. exit() also flushes library buffers; _exit() does not.

7.2 How a Shell Runs a Command and Gets Its Exit Status

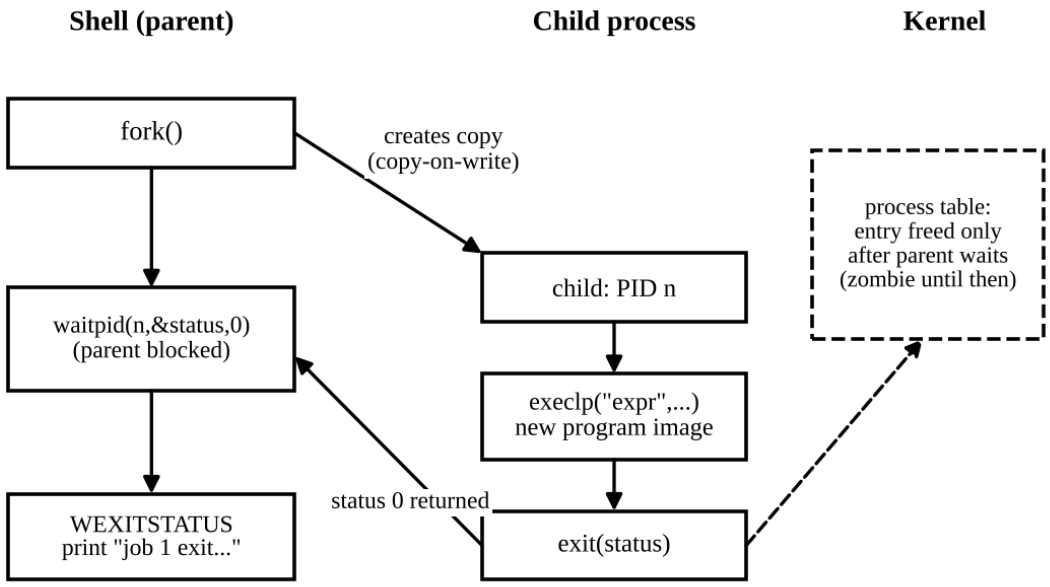


Figure 16: Process diagram of the shell running a command (fork, exec, wait, exit)

1. The shell reads a command and calls fork(); the kernel creates a child.
2. In the child, the shell calls exec() to replace itself with the command (expr).
3. In the parent, the shell calls waitpid() and sleeps until the child ends.
4. The command ends with exit(status); the kernel stores the status and wakes the parent.
5. waitpid() returns the status; the shell extracts it with WEXITSTATUS (0 means success) and frees the process-table entry.

7.3 Zombie and Orphan Processes

Table 33: Zombie and orphan processes

Term	Meaning	How the OS handles it
Zombie	A child that has ended but whose parent has not yet called wait(); its process-table entry (PID and exit status) remains	The entry is released when the parent calls wait(); if the parent ends, init/systemd adopts the zombie and reaps it
Orphan	A child that is still running after its parent has ended	The kernel re-parents it to init (PID 1) or a subreaper, which calls wait() when it ends

7.4 Why fork() Followed by exec() Instead of One "Create Process" Call

- Between fork() and exec() the child can change its own environment: redirect standard input and output, set up pipes, change directory and user, close descriptors. A single create-process call would need many parameters for all these options.
- The parent shell is not changed by these settings, because they are made in the child only.
- Copy-on-write makes fork() cheap: pages are copied only when written, and exec() discards the copy immediately.

7.5 Complete the Code

TODO 1 was completed by creating the child with fork(), replacing the child by expr with execlp() and exiting with 127 if exec fails, and in the parent waiting with waitpid() and printing the status with WEXITSTATUS. Declarations of pid and status, which the original code uses later, were added in the TODO region.

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   pid_t pid;
   int status;
   printf("shell: running job 1\n");
   fflush(stdout);  /* empty the buffer before fork() */
   // TODO 1: Create a child with fork(). In the child, replace the program with
   //  execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
   //  fails. In the parent, wait for the child with waitpid() and print "job 1 exit
   //  status N" using WEXITSTATUS
   pid = fork();
   if (pid < 0) { perror("fork"); return 1; }
   if (pid == 0) {  /* child */
      execlp("expr", "expr", "6", "*", "7", (char *)NULL);
      exit(127);  /* reached only if exec failed */
   }
   waitpid(pid, &status, 0);  /* parent */
   printf("job 1 exit status %d\n", WEXITSTATUS(status));

   printf("shell: running job 2\n");
   fflush(stdout);
   pid = fork();
   if (pid == 0) _exit(3);  /* a child that fails with code 3 */
   waitpid(pid, &status, 0);
   printf("job 2 exit status %d\n", WEXITSTATUS(status));
   return 0;
}
```

```
Terminal - Task 7: build and run q7_shell
$ gcc -std=c11 -Wall -Wextra -o q7_shell q7_shell.c
$ ./q7_shell
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Figure 17: Compilation and output of the completed shell program

The output matches the expected output: expr prints 42 (6 × 7), job 1 exits with status 0 and job 2, which calls _exit(3), exits with status 3.

7.6 Why stdout Is Flushed Before Each fork()

When output goes to a terminal the C library line-buffers it, but when it goes to a file or pipe it is fully buffered. The text printed before fork() may still be waiting in the program's memory buffer. fork() copies the whole address space, including this buffer, so the child would also contain the unwritten text and write it again when it flushes. fflush(stdout) before fork() empties the buffer first, so each line is written once.

Experiment: the second fflush(stdout) was removed, _exit(3) was changed to exit(3) and the output was redirected to a file.

```
Terminal - Task 7: buffering experiment (output redirected to a file)
$ ./q7_dup > out_dup.txt
$ cat out_dup.txt
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Figure 18: Output of the modified program redirected to a file

Now "shell: running job 2" (and also "job 1 exit status 0") appears twice. At the second fork() the buffer still held "job 1 exit status 0" and "shell: running job 2" because they had not been flushed. The child inherited a copy of this buffer, and exit(3) flushes the library buffers, so the

child wrote both lines; the parent wrote them again when it ended. With _exit(3) the child ends without flushing, so no duplicates appear.

7.7 Alternative Concept and Complexity Analysis: posix_spawn() or Threads

posix_spawn() combines creation and execution in one call and avoids copying the parent's memory (it is typically implemented with vfork-style sharing). Threads start another function inside the same process, not another program. The time to start 1,000 commands (/bin/true) and 1,000 empty tasks was measured with bench_proc.c.

```
Terminal - Task 10: creating 1,000 processes versus 1,000 threads

$ ./bench_proc fork ; ./bench_proc forkbig ; ./bench_proc thread
1000 x fork()+exit+wait         :     54.6 ms
1000 x fork() of 200 MB process :   1764.1 ms
1000 x pthread_create+join      :      8.8 ms
$ ./bench_proc forkexec ; ./bench_proc vforkexec ; ./bench_proc spawn
1000 x fork()+exec(true)+wait   :    650.5 ms
1000 x vfork()+exec(true)+wait  :    519.0 ms
1000 x posix_spawn(true)+wait   :    565.0 ms
```

Figure 19: Measured time for 1,000 process creations, thread creations and command starts

Table 34: Starting 1,000 commands: fork+exec, vfork+exec, posix_spawn and threads

Method	**Time for 1,000 (ms)**	**Memory copied or shared**	**Isolation**
fork() + exec()	650.5	Page tables copied, pages copy-on-write	Full (separate address space)
vfork() + exec()	519.0	Nothing copied (parent suspended)	Full after exec
posix_spawn()	565.0	Nothing copied	Full
Threads (pthread_create, empty function)	8.8	All shared (same address space)	None: one fault or bug affects all

The three process methods are within about 25 percent of each other because the cost of loading the new program (exec) dominates, so posix_spawn is only modestly faster. The cost of fork() alone grows with the size of the parent: 1,000 forks of a small process took 54.6 ms, but 1,000 forks of a process with 200 MB of touched memory took 1,764 ms, 32 times longer, because the page tables must be copied. Therefore fork() is O(M) in the parent's mapped memory M,

while vfork() and posix_spawn() are O(1) in M. Threads are by far the fastest (6 times faster than fork in this test) but give no isolation, which is unacceptable between commands on a patient-record server.

Evidence provided: a working shell-like program, a process diagram and a reasoned evaluation with measurements.

TASK 8 – Evaluate CPU States and Protection

Bloom's Level: K5 – Evaluate. This task evaluates how the hardware protects the hospital system.

8.1 User Mode, Kernel Mode and the Mode Bit

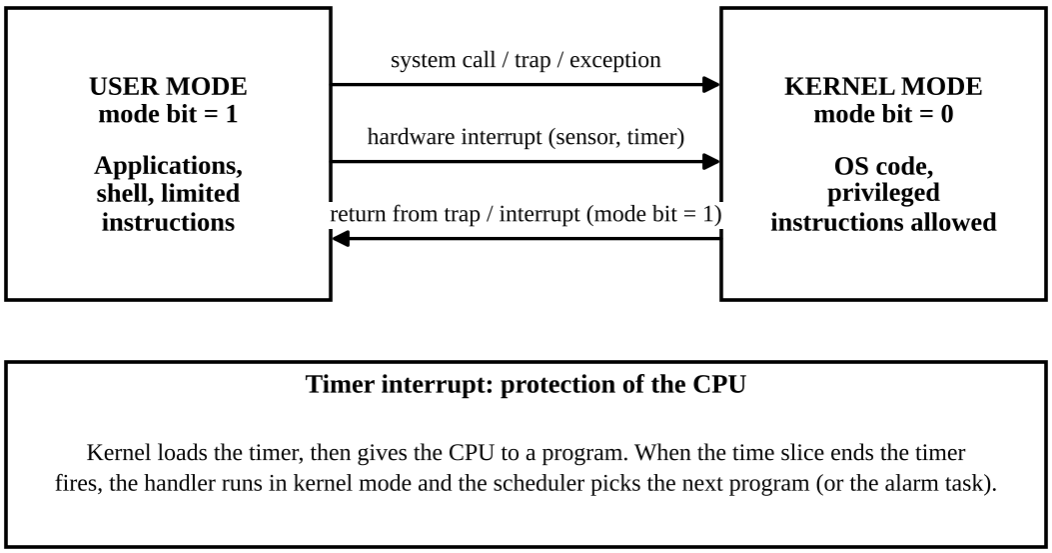


Figure 20: CPU modes, mode transitions and the timer interrupt

The CPU has a mode bit: 0 means kernel mode and 1 means user mode. Privileged instructions (disable interrupts, access I/O ports, change the page-table base, halt the CPU, set the timer) can run only in kernel mode. If a user program executes one, the hardware raises an exception and the OS stops the program. User programs reach privileged services only by system calls, which switch to kernel mode at a fixed entry point and return to user mode afterwards.

8.2 Interrupts, Traps and the Timer Interrupt

Table 35: Interrupts, traps and the timer interrupt

Event	Source	Example in the hospital system
Hardware interrupt	An external device signals the CPU asynchronously	Sensor value outside limits on a bedside monitor; MRI scanner finished a block
Trap (software interrupt)	A program deliberately requests the OS	A system call such as read() or a call to raise an alert
Exception	An error in the program or an illegal action	Divide by zero, invalid memory address, privileged instruction in user mode
Timer interrupt	A hardware timer expires	Ends the time slice so the OS regains control and runs the scheduler, preventing a program from monopolising the CPU

8.3 If Any Program Could Disable Interrupts

A single faulty or malicious program could execute the instruction that disables interrupts. The sensor and timer interrupts would then never reach the monitor's software: the alarm handler would not run, the timer would not return control to the OS, the program would run forever and the monitor would not detect or report an abnormal reading. The patient would be unmonitored, with no software means of recovery except an external hardware watchdog. This is why the instruction is privileged.

8.4 How the Hardware Stops One Program from Reading Another Program's Memory

- **Memory-management unit (MMU):** every address a program uses is virtual and is translated through that process's page table to a physical address. A process has no page-table entries for another process's memory.
- **Permission bits:** each page entry has read, write and execute bits, and a user/supervisor bit, so user code cannot touch kernel pages.
- **Violation:** an access outside the process's pages causes a page-fault exception; the OS terminates the process (segmentation fault).

- **Older scheme:** base and limit registers check that each address is between the base and base + limit; changing them is privileged.

8.5 Alternative Concept and Complexity Analysis: Polling Instead of Interrupts on the Monitors

With polling, the CPU reads the sensor in a loop every T seconds. Worst-case delay before an abnormal reading is noticed: T. With interrupts the delay is the interrupt latency, typically 5–50 µs (assumed typical range). If one poll costs c = 2 µs of CPU time (assumed), the fraction of CPU time spent polling is c / T.

Table 36: Polling compared with interrupts for a bedside monitor (assumed typical values)

Measure	**Polling, T = 10 ms**	**Polling, T = 1 ms**	**Polling, T = 100 µs**	**Interrupt-driven**
Worst-case detection delay	10 ms (equals the whole deadline)	1 ms	100 µs	About 5–50 µs
CPU time spent polling (c = 2 µs)	0.02 %	0.2 %	2 %	0 % (CPU idle or sleeping)
Polls per second	100	1,000	10,000	Only when an event occurs
Power	CPU cannot sleep deeply	CPU cannot sleep deeply	CPU cannot sleep deeply	CPU sleeps between events

To meet the 10 ms requirement with a safety margin, polling would need T well below 10 ms, which keeps the CPU permanently awake and costs O(1/T) CPU time and power for every second of operation. Interrupts cost time only when an event happens (O(number of events)) and give the smallest delay. Interrupts are therefore chosen for the monitors.

Evidence provided: a reasoned explanation of CPU modes, interrupts and protection applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems

Bloom's Level: K5 – Evaluate. This task evaluates how data moves between storage levels and devices.

9.1 Levels of the Memory Hierarchy

Table 37: Memory hierarchy with typical access times and sizes

Level	Typical access time	Typical size	Volatile?
Registers	Less than 1 ns	Less than 1 KB	Yes
Cache (L1, L2, L3)	1–30 ns	KB to tens of MB	Yes
Main memory (RAM)	About 100 ns	GB to TB	Yes
SSD	About 100 µs	100s of GB to TB	No
Hard disk	About 5–10 ms	TB	No
Network storage	About 0.5–10 ms or more	Up to PB	No

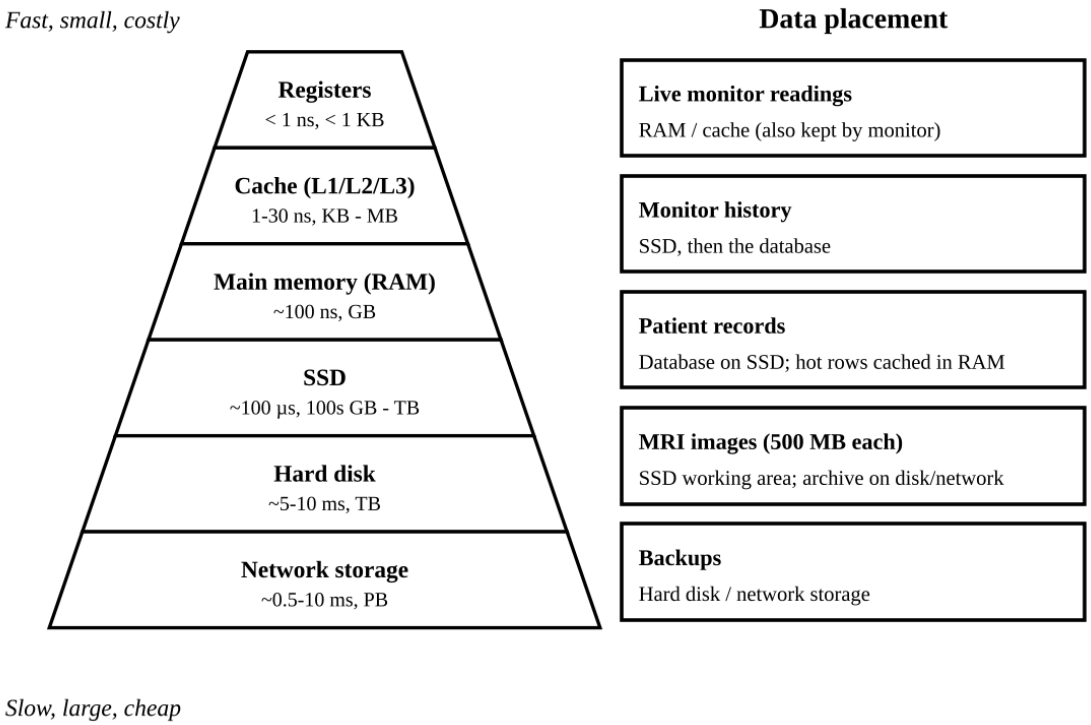


Figure 21: Memory hierarchy and placement of the hospital's data

9.2 Where the Hospital's Data Should Be Kept

Table 38: Data placement plan

Data	**Placement**	**Reason**
Live monitor readings	RAM (and cache) of the monitor	Must be examined within milliseconds; volatile data is acceptable because values are streamed to the server.
Monitor history and alarms	Local flash/SSD, then the database	Must survive power loss but is accessed rarely.
Patient records	Database on SSD, replicated on the cluster's shared storage, frequently used rows cached in RAM	Must be durable, fast (about 100 µs) and never lost.
MRI images (about 500 MB each)	SSD working area during processing; long-term archive on hard disk or network storage	Too large to keep in RAM for long; SSD gives speed during processing and cheap disk storage for the archive.
Backups	Hard disk and network storage at another site	Large, cheap and sequentially written.

9.3 Programmed I/O, Interrupt-Driven I/O and DMA

Table 39: I/O methods and suitability for the MRI scanner

Method	How it works	CPU involvement	Suitability for the MRI scanner
Programmed I/O (polling)	CPU loops checking the device status and copies each word	Complete: CPU busy for the whole transfer	Unsuitable for 500 MB images
Interrupt-driven I/O	Device interrupts when a word or small unit is ready; the CPU copies it	Interrupt handling for every word	Wasteful for large images
DMA	A DMA controller moves whole blocks between the device and memory; one interrupt at the end of a block	CPU sets up the transfer and handles one interrupt per block	Best: used for the scanner

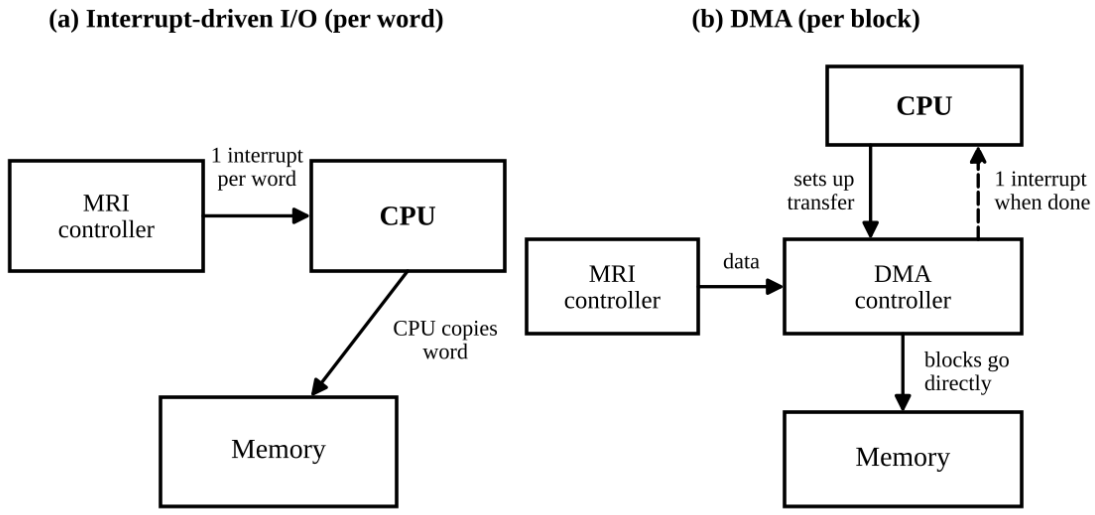


Figure 22: Interrupt-driven I/O compared with DMA

9.4 Effect of Caching on Record Look-up

The effective access time is EAT = h × t(cache) + (1 − h) × t(storage), where h is the hit ratio. Taking t(cache) = 0.1 µs (RAM) and t(storage) = 100 µs (SSD, from the table above), for a record cache with h = 0.99:

EAT = 0.99 × 0.1 + 0.01 × 100 = 0.099 + 1.0 = 1.099 µs, compared with 100 µs without a cache, which is about 91 times faster. Even with h = 0.90 the EAT is 0.09 + 10 = 10.09 µs (about 10 times faster). Caching works because patient look-ups show locality: the same active patients' records are used again and again during a shift.

9.5 Alternative Concept and Complexity Analysis: Interrupt-Driven I/O (One Interrupt per Word) for the MRI Scanner

Assumptions: image size 500 MB = 500 × 1,048,576 = 524,288,000 bytes; word size 4 bytes; handling one interrupt costs 2 µs of CPU time (assumed); DMA block size 64 KB.

Table 40: Transfer of a 500 MB image: interrupt-driven I/O compared with DMA

Measure	**Interrupt per word**	**DMA (64 KB blocks)**
Number of interrupts	524,288,000 / 4 = 131,072,000	524,288,000 / 65,536 = 8,000
CPU time for interrupts (2 µs each)	131,072,000 × 2 µs = 262.1 s	8,000 × 2 µs = 16 ms
Complexity	O(n / word size)	O(n / block size)
Memory buffers needed	Only a word-size register, plus the 500 MB destination in memory	A 64 KB buffer (128 KB if double-buffered) plus the 500 MB destination
CPU available for other work	Almost none during the transfer	Almost all

DMA reduces the interrupt count by a factor of 16,384 (65,536 / 4) and the CPU time from about 262 seconds to 16 milliseconds, at the cost of a small extra buffer. DMA is therefore chosen for the MRI scanner.

Evidence provided: a justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare

Bloom's Level: K5 – Evaluate. The behaviour of the programs was measured on a Linux system (Table 2 describes the environment).

10.1 System Calls for Different Buffer Sizes (strace -c)

The copy program was run with buffer sizes of 1, 512 and 4096 bytes on the 10,000-byte file and counted with the strace -c style tool (screenshots in Task 6.3).

Table 41: System calls made by the copy program for different buffer sizes

Buffer	Total read() calls shown	Total write() calls shown	Total system calls
1 byte	10,002	10,004	20,047
512 bytes	22	24	87
4096 bytes	5	7	53

The total falls from 20,047 to 53 calls, a reduction of 99.7 percent, when the buffer grows from 1 byte to 4096 bytes. (The values include the one loader read and the few start-up and message calls.)

10.2 Time in User Mode and Kernel Mode (time Command)

The time command reports real (elapsed), user (CPU time in user mode) and sys (CPU time in kernel mode). The 4 MB copy was timed with each buffer size.

Terminal - Task 10: copying a 4 MB file with different buffer sizes (time)

```
$ for b in 1 16 64 512 4096 65536 1048576; do time ./bench_copy big4m.dat $b; done
bufsize       1 :   4194304 read() calls,   1757.1 ms  real 0m1.759s user 0m0.402s sys 0m1.336s
bufsize      16 :    262144 read() calls,     89.7 ms  real 0m0.091s user 0m0.021s sys 0m0.070s
bufsize      64 :     65536 read() calls,     27.8 ms  real 0m0.030s user 0m0.008s sys 0m0.020s
bufsize     512 :      8192 read() calls,      6.5 ms  real 0m0.009s user 0m0.000s sys 0m0.007s
bufsize    4096 :      1024 read() calls,      2.0 ms  real 0m0.004s user 0m0.003s sys 0m0.000s
bufsize   65536 :        64 read() calls,      1.9 ms  real 0m0.004s user 0m0.000s sys 0m0.003s
bufsize 1048576 :         4 read() calls,      1.3 ms  real 0m0.003s user 0m0.000s sys 0m0.002s
```

Figure 23: time output for each buffer size of the 4 MB copy

Table 42: Real, user and kernel time for the 4 MB copy

Buffer (bytes)	Real (s)	User (s)	Sys (s)	Sys as share of CPU
1	1.759	0.402	1.336	77 %
16	0.091	0.021	0.070	77 %
64	0.030	0.008	0.020	71 %
512	0.009	0.000	0.007	100 %
4,096	0.004	0.003	0.000	0 %
65,536	0.004	0.000	0.003	100 %
1,048,576	0.003	0.000	0.002	100 %

For small buffers the kernel time (system calls and copying) is about three-quarters of all CPU time, because each tiny read() and write() pays the mode-switch cost. For buffers of 4 KB and above the whole run takes only a few milliseconds, which is at the resolution of the time command, so the rows for large buffers are only indicative.

10.3 Creating 1,000 Processes and 1,000 Threads

```
Terminal - Task 10: user and kernel time (time command)

$ time ./bench_proc fork
1000 x fork()+exit+wait         :     40.0 ms

real 0m0.041s
user 0m0.022s
sys 0m0.017s
$ time ./bench_proc thread
1000 x pthread_create+join      :      7.2 ms

real 0m0.008s
user 0m0.004s
sys 0m0.004s
```

Figure 24: time command for 1,000 fork() calls and for 1,000 thread creations

Table 43: Creating 1,000 processes and 1,000 threads

Operation	**Time (ms)**	**Relative**
1,000 × fork() of a small process (exit + wait)	54.6 (40.0 in the second run)	1.0
1,000 × pthread_create + join	8.8 (7.2 in the second run)	0.16 (6.2 times faster)
1,000 × fork() of a process with 200 MB touched	1,764.1	32 times slower than a small fork

10.4 Do the Measurements Support the Earlier Judgements?

Table 44: Comparison of measurements with expectations

Earlier judgement	Measurement	Supports?
A system call is far more expensive than a function call (Task 6)	100.6 ns against 0.4 ns: about 233 times	Yes
Calls fall as n / b with buffer size (Task 5)	4,194,304 / b read() calls matched exactly for all seven sizes	Yes
A 4 KB buffer or larger removes most overhead (Task 6)	Time fell from 1,757 ms to 2.0 ms at 4096 bytes and then flattened	Yes
Microkernel IPC costs more than a system call (Task 3)	IPC round trip 2,471 ns, 24.6 times a system call	Yes; still negligible against 10 ms
Threads are cheaper than processes (Task 7)	8.8 ms against 54.6 ms for 1,000 creations	Yes
fork() cost grows with process size (Task 7)	200 MB process: 1,764 ms against 54.6 ms	Yes
posix_spawn is much faster than fork+exec (expectation)	565 ms against 650 ms, only about 13 percent faster	Partly: exec dominates the cost
Larger buffers keep giving gains (expectation)	Gain from 64 KB to 1 MB is small (1.9 ms to 1.3 ms) for a cached 4 MB file	Partly: diminishing returns

10.5 Alternative Concept and Complexity Analysis: Threads Instead of Processes

Creating a thread does not create a new address space; the thread receives only its own stack and register set, while code, heap, global data and open files are shared. Creating a process copies the page tables and file-descriptor table (O(M) in mapped memory) and the process later needs its own page tables and kernel structures.

Table 45: 1,000 threads compared with 1,000 processes

Measure	**1,000 processes (fork)**	**1,000 threads**
Creation time (measured)	54.6 ms	8.8 ms
Complexity of creation	O(M), M = mapped memory of the parent	O(1) in M (one stack and one thread descriptor)
Memory per task	Own page tables, kernel structures and (copy-on-write) pages	Own stack (about 8 MB of virtual space by default, only touched pages are resident) and a small descriptor
What is shared	Nothing by default (pipes or shared memory must be set up)	Address space, heap, global variables, open files
Protection	A fault is confined to one process	A fault or overwritten memory can affect all threads

The difference is explained by what each shares: threads share the address space, so there is nothing to copy; processes are isolated, so the kernel must create a private copy of the memory-management structures.

Evidence provided: measured results in tables, compared with expectations and with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan

Bloom's Level: K5 – Evaluate. The complete operating-system plan for the hospital is recommended, based on the analysis and measurements above.

11.1 Recommended Plan

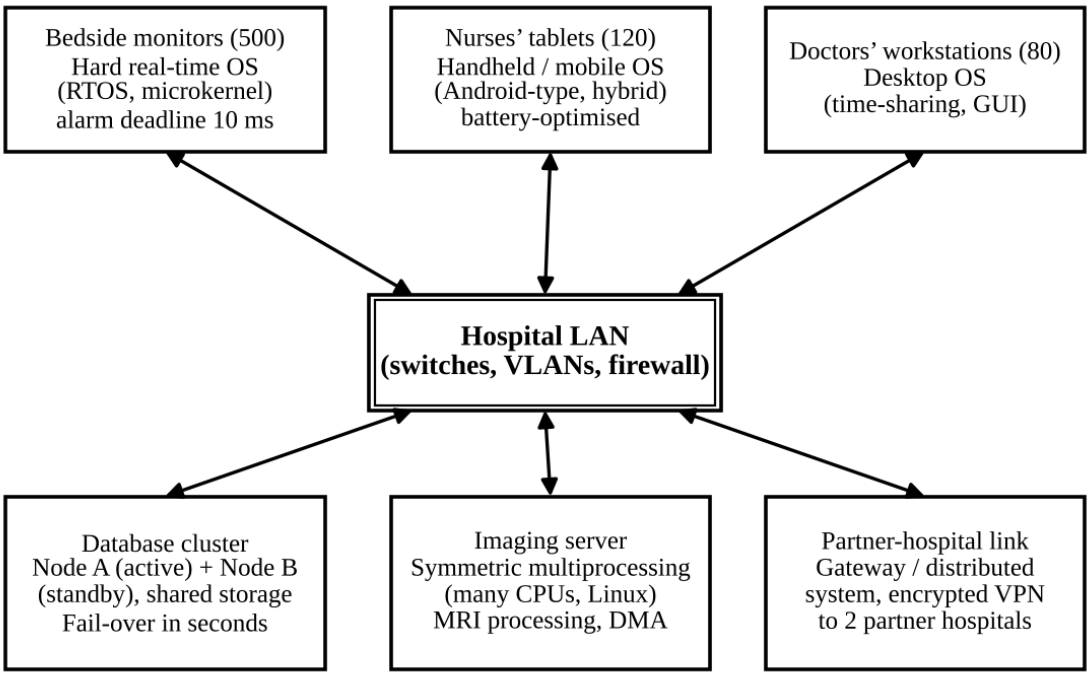


Figure 25: Recommended operating-system plan for the hospital

Environment	OS type and structure	Services	Protection	I/O method
Bedside monitors	Hard real-time OS; microkernel RTOS (or minimal RTOS)	Task scheduling, sensor I/O, network, error detection; no GUI, no multi-user, no general file system	Hardware modes, memory protection (MPU), signed firmware, authenticated network	Interrupt-driven sensor input; small DMA for network
Nurse tablets	Handheld mobile OS; layered/hybrid structure	Touch UI, power management, application sandbox, encrypted storage, wireless	Per-app sandbox, device encryption, login, remote wipe	Interrupt-driven touch and wireless, DMA for storage
Doctors' workstations	Desktop OS; hybrid or modular kernel	GUI, multitasking, file system, printing, communication	User accounts, role-based access, disk encryption	Interrupt and DMA for disks and display
Database server	Clustered, fail-over; Linux with modular monolithic kernel	File system with journal, process and resource scheduling, communication, accounting and audit	Authentication, file permissions, encryption at rest and in transit, audit logs	DMA for disk and network; cache in RAM
Imaging server	Symmetric multiprocessing Linux; modular kernel	Many-core scheduling, large-memory management, file system	Authentication and access control, encrypted store	DMA from the scanner to memory

Environment	OS type and structure	Services	Protection	I/O method
Partner link	Distributed system through a secure gateway	Authentication, encrypted communication, record replication and conflict handling	TLS/VPN, mutual authentication, access rules	Interrupt-driven network I/O with DMA

Table 46: Type, structure, services, protection and I/O method for each environment

11.2 Evidence from the Analysis and Measurements

- A guaranteed 10 ms alarm needs a hard real-time OS: a general-purpose scheduler gives about $(N - 1) \times q$ worst-case delay (Task 1).
- The microkernel's extra message cost is 2.5 µs against 0.1 µs for a system call (measured), only 0.025 percent of the deadline (Task 3).
- A reduced handheld OS gives 10 hours of battery life against 4.6 hours for a desktop OS (Task 4).
- System calls cost about 233 times a function call; buffer sizes of 64 KiB cut calls by a factor of 65,536 and time by a factor of 925 (Tasks 6 and 10).
- A cluster with fail-over raises availability from 99.9 to 99.9999 percent (Task 2).
- DMA reduces the CPU time of a 500 MB transfer from 262 s to 16 ms (Task 9).

11.3 Limitations and Risks of the Plan

Table 47: Limitations and risks

Risk or limitation	Effect	Mitigation
Five OS families to maintain	More skills, more patch processes, higher administration effort	Standardise on few platforms (one RTOS, one mobile OS, Linux for servers); central patch management
Failure of the shared storage or network of the cluster	Fail-over does not help if the common component fails	Mirrored storage, redundant network paths, off-site backup
Microkernel and IPC overhead	Slower service requests on monitors	Measured cost is negligible (2.5 µs)
Security of lost tablets or compromised workstations	Exposure of patient data	Encryption, remote wipe, least privilege, audit logs
Updates to the real-time monitors	A bad update could stop alarms	Signed, staged roll-out with rollback and a hardware watchdog
Timings measured on a small virtual machine	Absolute numbers differ on real hardware	Ratios and counts are reliable; re-measure on target hardware

11.4 Is the Plan Suitable for a Group of Three Hospitals?

The group would have about 1,500 monitors, 360 tablets and 240 workstations. Bedside monitors scale linearly because each is independent and keeps its alarm logic locally, so growth does not affect the 10 ms deadline. The database cluster would need more nodes (the data is partitioned by hospital or replicated) and the imaging servers can be replicated per hospital. The partner link becomes a distributed service among three hospitals. The plan therefore remains suitable, provided that central management of updates, identity and keys is introduced.

11.5 Alternative Concept and Complexity Analysis: Virtual Machines on Two Large Servers

Table 48: Separate physical machines compared with virtual machines on two large servers

Measure	Separate physical machines	Virtual machines on two servers
Response time for the monitors	Direct: hardware interrupt to alarm, below 1 ms	The bedside devices remain physical; any alarm server inside a VM adds hypervisor scheduling delay and jitter (typically tens of µs to milliseconds), so the real-time part must stay outside the VMs or use a real-time hypervisor
Memory overhead (space)	No duplication	Each VM carries its own guest OS (about 0.5–2 GB); memory over-commitment risks swapping
Processor overhead (time)	None	About 2–10 percent (typical) from virtualised privileged operations and I/O
Reliability	A hardware fault affects one environment	A host failure stops all VMs on it, so two-host fail-over and live migration are needed
Administration and cost	Many machines	Fewer machines, central management and quick recovery of VM images

Virtualisation is appropriate for the workstations' back-end services and for the database and records systems, but it is not recommended for the real-time alarm path. The recommended hybrid keeps dedicated hardware for monitors and imaging and may use virtualisation for the remaining servers.

Evidence provided: a technically justified plan supported by analysis and measurements.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Bloom's Level: K5 – Evaluate. **Changed requirement:** the hospital adds remote monitoring devices in patients' homes that send readings over the internet, and moves patient records to a cloud service shared with the partner hospitals. The original solution is re-evaluated and may be retained, modified or redesigned.

A. Impact Analysis

Area	Original design	Effect of the change	Affected?
OS type for patient devices	Hard real-time OS on bedside monitors inside the LAN	New devices are outside the hospital, with unreliable internet and no local staff: they need an embedded real-time OS with local alarms	Yes: new type
Database server	Clustered local database	Records move to a cloud service shared by three organisations: a distributed system	Yes: major
Partner link	Private link to two hospitals	Replaced by shared cloud access	Yes
Services	Local file system and LAN communication	Remote authentication, encrypted communication, remote updating, device management	Yes: added
Protection	Protection inside a trusted LAN	Internet exposure: authentication of devices and users, encryption in transit and at rest, per-patient access, audit, key management	Yes: strengthened
Tablets, workstations, imaging	Local use	Access the cloud records but otherwise unchanged	Slightly (new client access)
Bedside monitors in the hospital	Hard real-time OS	Unchanged	No

Table 49: Operating-system types, services and protection mechanisms affected by the changed requirements

B. Evaluation of the Original Design

Table 50: Can the original plan support home devices and cloud records?

Criterion	Evaluation of the original design	Supported?
Response time	Alarms stay local to each device in a real-time OS, so deadlines are met. But a path that depends on the internet to the hospital has unbounded delay (seconds), so home alarms cannot depend on the hospital.	Only with local alarm logic on the home device
Reliability	The local cluster gives high availability, but the home link may fail; and a cloud dependency creates a new single point of failure.	No: needs store-and-forward and replicas
Security	The original design assumes a trusted LAN and a private partner link. Home devices and the cloud are reachable from the internet, so the attack surface is much larger.	No: needs new protection
Scalability	Data from thousands of homes exceeds the capacity of the local database and its cluster design.	No: needs a distributed service

C. Adaptation or Redesign

Decision: MODIFY. The in-hospital design (real-time monitors, mobile tablets, desktop workstations, SMP imaging server) is retained because its requirements have not changed. It is modified by adding new components.

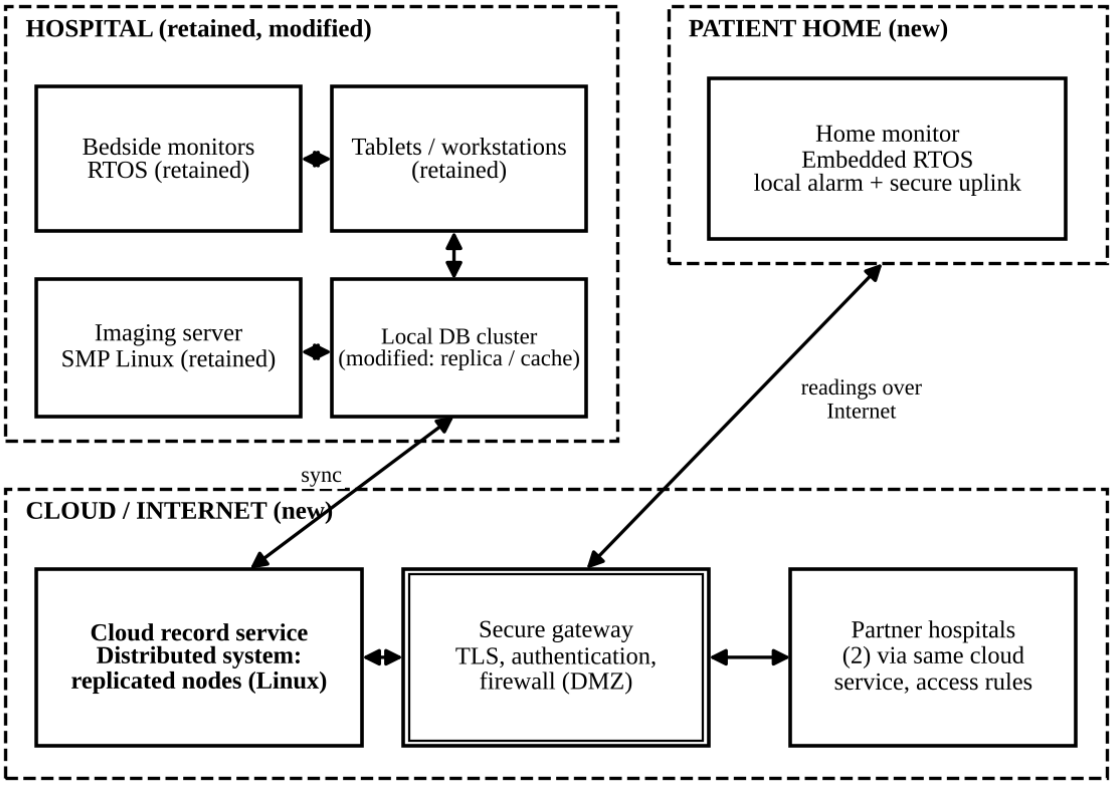


Figure 26: Revised operating-system plan with home devices and a cloud record service

Table 51: Revised plan

Component	**Revised design**	**Reason**
Home monitoring devices (new)	Embedded real-time OS (microkernel/RTOS); local alarm; encrypted, authenticated uplink; store-and-forward buffer; signed over-the-air updates	Local alarms regardless of the internet; small, secure and reliable
Cloud records (new)	Distributed system: replicated nodes running Linux across sites, with access control per patient and audit	Shared by three hospitals; no single point of failure; scalable
Secure gateway (new)	Firewalled gateway (DMZ) terminating TLS, authenticating devices and partners	Protects the hospital network from the internet
Local database cluster (modified)	Kept as a local replica/cache with fail-over	Hospital continues to work if the cloud or link fails
Bedside monitors, tablets, workstations, imaging server (retained)	Unchanged OS types; clients use the cloud through the gateway	Requirements unchanged

D. Evidence-Based Defence

Options were scored from 1 (poor) to 5 (best) against weighted criteria. The scores are engineering judgements based on the analysis above.

Table 52: Decision matrix for retain, modify and redesign

Criterion (weight)	**Retain**	**Modify**	**Redesign (full cloud)**
Safety and response time (30)	5	5	4
Reliability and availability (25)	2	4	5
Security (20)	1	4	4
Cost (10)	5	3	1
Effort and time to change (15)	5	3	1
Weighted score (out of 5)	3.45	4.05	3.50

Weighted score = sum of weight × score divided by 100; for Modify: (30×5 + 25×4 + 20×4 + 10×3 + 15×3) / 100 = 4.05.

- **Evidence from earlier tasks:** the measured cost of a message exchange (2.5 µs) and the 10 ms alarm budget show that a real-time microkernel is appropriate for the home devices as well; the measured expense of system calls and process creation shows that the cloud nodes should use buffered and batched communication; and the 99.9 to 99.9999 percent availability analysis supports replication across nodes.
- **Trade-offs:** MODIFY costs more than RETAIN (new gateway, cloud service, device management) and adds more operating-system families to maintain; but retaining the original design leaves the home devices and the cloud insecure and unreliable. A full REDESIGN gains availability but costs most and moves the alarm-critical real-time components away from proven hardware, adding risk and time.
- **Risks that remain:** cloud provider outage (reduced by the local replica), internet loss at home (local alarm and store-and-forward), key and update management for thousands of devices, and compliance with privacy rules for shared records.

Final decision: retain the real-time monitors, tablets, workstations and imaging server; modify the database and partner link into a replicated cloud service behind a secure gateway; add embedded real-time OS devices at home with local alarms.

Evidence provided: impact analysis, evaluation of the original design, revised plan and an evidence-based defence integrating the changed requirements.